

Cloud Infrastructure Security

By

Dr Mohit Mathur

Virtualization security

Virtualization security focuses on protecting hypervisors and virtual machines (VMs) from various threats and vulnerabilities, ensuring a secure and stable virtual environment. Hypervisors, the software that manages VMs, are a critical target, as their compromise could compromise all VMs on the host. Security measures include strong access controls, regular updates, and advanced security solutions.

Virtual Machine Security in Cloud

- As cloud computing becomes more popular, virtual machines (VMs) are a key component of cloud infrastructure. It allows a business to run several OS and programs on a physical server. Thus, it provides:
 - Flexibility
 - Scalability
 - Cost benefits
- A significant security challenges accompany these advantages. Protecting virtual machines in the cloud is necessary for data protection, compliance with the rules, and countering cyber risks.
- Security in virtual machines involves safeguarding the virtualized settings in which VMs function. These settings include the hypervisor, the software layer overseeing VMs, the physical infrastructure underneath, and the VMs. Security in this situation needs to cover typical IT security issues like safeguarding data and controlling access and unique obstacles related to virtualized and cloud settings.

Key Aspects of Virtual Machine Security

1. Hypervisor Security
2. Isolation of Virtual Machines
3. Data Protection
4. Identity and Access Management (IAM)
5. Monitoring and Logging

1. Hypervisor Security

- The hypervisor is the basic virtualization layer that helps create and manage virtual machines. A hypervisor is software that enables a single host device to run multiple virtual machines (VMs). It is the protection layer between the physical hardware and the VMs running on it. Hence its security is paramount. In case of a successful attack on the hypervisor, the hackers have full control of all the VMs on the host. It results in potentially disastrous outcomes. Numerous methods exist to help protect hypervisors. Monitoring and network security tools enable administrators to track their virtual environments and detect any unusual behavior or attacks. Administrators can also use tools such as firewalls to keep hypervisors secure.
- **Hypervisor security include:**
 - Regular Patching and Updates Updating the hypervisor software regularly with the latest security patches is crucial. It helps to defend against identified vulnerabilities.
 - Access Control Limit access to the hypervisor management interface to authorized personnel. It can help preclude change or manipulation by unauthorized users and possible security breaches.
 - Multi-Factor Authentication (MFA) Security will be improved if MFA is implemented for hypervisor access. It creates an additional barrier for attackers to overcome.

2. Isolation of Virtual Machines

A significant security risk in virtualization is when a VM gets compromised. It attacks other VMs in the same host through a virtual network interface card. Maintaining adequate isolation among VMs is essential to avoid potential lateral movement.

VM isolation include:

- **Network Segmentation** :Separating VMs into various network segments according to their purpose and security level can restrict the propagation of a cyber assault. For instance, minimizing risk involves segregating VMs containing sensitive data from those supporting public-facing applications.
- **Virtual Firewalls** Virtual firewalls between virtual machines (VMs) can monitor and regulate traffic. Thus, it stops unauthorized access & data breaches.
- **Security Groups and Access Control Lists (ACLs)** Employ security groups and ACLs to create and uphold rules that limit VM communication according to requirements.

3.Data Protection

- Data protection is a critical priority due to the sensitive nature of data handled by virtual machines in the cloud. Safeguarding data while it is stored, transferred, and processed is crucial for preserving informations:
- Confidentiality
- Integrity
- Availability

4. Identity and Access Management (IAM)

- Restricting access to the VMs and defining the actions allowed for the VMs are important aspects of security. IAM policies must be strong and follow the principle of least privilege. It ensures users are only granted the necessary permissions for their tasks.
- IAM include:
 - **Role-Based Access Control (RBAC)** Utilize RBAC to allocate permissions according to user roles. It reduces the chance of unauthorized entry.
 - **Regular Audits and Monitoring** Monitor access logs constantly and conduct regular audits to identify and address any unusual activities. Automated notifications can detect possible breaches at an early stage.
 - **MFA (Multi Factor Authentication)for VM Access** Like hypervisor access, MFA should be required to access VMs, particularly for administrative tasks.

5. Monitoring and Logging

- It is crucial to continuously monitor virtual machine activity and keep thorough logs to detect security incidents and respond appropriately. Monitoring software can assist in detecting abnormal behavior patterns that could signal a security breach or other security problem.

Monitoring and logging include:

- **Intrusion Detection Systems (IDS)** Implement IDS tools to identify and send notifications about potentially harmful behavior in virtual machines or the hypervisor layer.
- **Regular Security Audits** Perform routine security checks and vulnerability assessments to discover and fix possible weaknesses.

Challenges in Virtual Machine Security

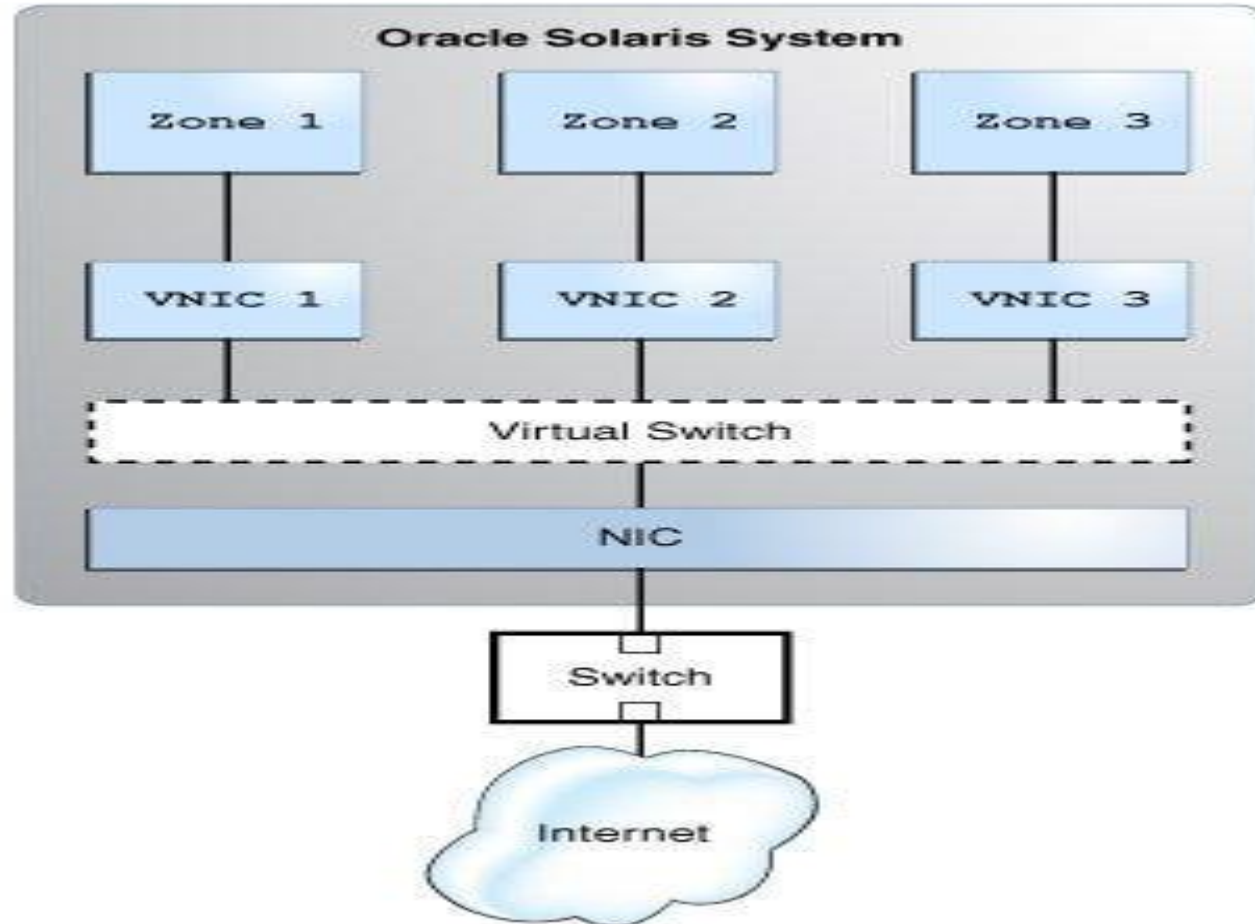
- 1. Complexity of Multi-Tenant Environments** Multiple tenants in cloud environments frequently use the same physical infrastructure, which can lead to worries regarding the safety and confidentiality of data. Ensuring that one tenant's data and operations do not disrupt another's is difficult yet crucial.
- 2. Dynamic Nature of Cloud Environments** Cloud environments are dynamic; virtual machines are created and removed as required. The ongoing changes can create challenges in upholding consistent security measures and guaranteeing proper protection for all VMs.
- 3. Insider Threats** In cloud environments, the risk of insider threats from employees' malicious or careless actions is considerable. Ensuring IAM policies are upheld and watching for abnormal behavior can aid in reducing this risk.
- 4. Compliance and Regulatory Requirements** Compliance with industry standards and regulations may be challenging in cloud environments. Organizations need to ensure their VM security practices align with these requirements to prevent penalties and breaches.

Virtual Network

- A virtual network connects services and resources like virtual machines and database applications with each other and the rest of the internet via a secure, encrypted, and private network. The virtual network provides a framework that gives your enterprise cloud infrastructure substance.
- Here we connect our resources via wireless technology. This allows the reach of the network to be expanded as far as it needs to for peak efficiency, in addition to numerous other benefits.
- Virtual networking enables communication between multiple computers, virtual machines (VMs), virtual servers, or other devices across different office and data centre locations.

How a Virtual Network Works:

- The following figure shows the working of a virtual network and its components in a system.



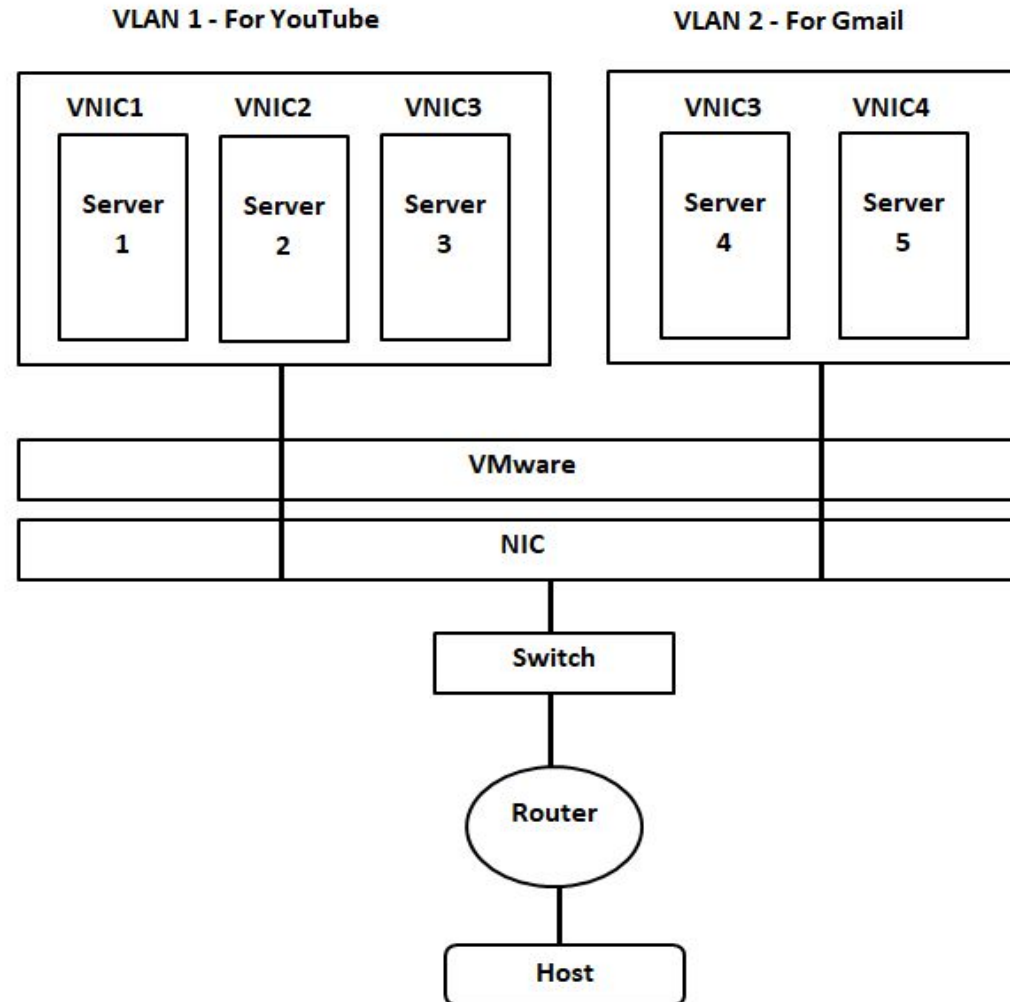
- The figure shows a single system with one NIC. The NIC is configured with three VNICs. Each VNIC is assigned to a zone. Zone 1, Zone 2, and Zone 3 are the three zones configured for use in the system. The zones communicate with each other and with the external network by using their respective VNICs. The three VNICs connect to the underlying physical NIC through the virtual switch. The function of a virtual switch is equivalent to the function of a physical switch as both provide connectivity to the systems.
- When a virtual network is configured, a zone sends traffic to an external host in the same way as a system without a virtual network. Traffic flows from the zone, through the VNIC to the virtual switch, and then to the physical interface, which sends the data to the network.
- The zones can also exchange traffic with one another inside the system if all the VNICs configured to the zones are part of the same VLAN. For example, packets pass from Zone 1 through its dedicated VNIC 1. The traffic then flows through the virtual switch to VNIC 3. VNIC 3 then passes the traffic to Zone 3. The traffic never leaves the system, and therefore never violates the Ethernet restrictions.
- Alternatively, you can create a virtual network based on the ether stub. Ether stubs are entirely software based and do not require a network interface as the basis for the virtual network.

Examples of Virtual Networks

Example of virtual networking is a **Virtual Local Area Network (VLAN)**. A VLAN is a subgroup of a network, which combines multiple network devices into one grouping, or domain, and partitions it off from the rest. VLANs improve network speed and performance by more efficiently routing traffic across these subgroups or domains. VLANs also allow significantly more control over network devices and traffic. Isolating certain data within a separate VLAN provides additional security benefits, especially to large networks, by making it more difficult for unauthorized monitoring or interference with the network. VLANs also eliminate the need to add new cabling or make big changes to network infrastructure.

Concept of Virtual Networking to host Google Services

- For the simplicity we are considering that google is giving two services:
- 1.YouTube and
- 2.Gmail.
- Let's consider VLAN 1 is handling the YouTube Service and VLAN 2 is handling the Gmail service.



Let's see how this services are working :

- **Different servers at different regions but in same VLAN**
- Here server 1, 2 and 3 are physically present at different regions but we have added them virtually in same VLAN using Virtual Networking. So if any server has stopped working due to some issues of that region then other servers can provide the services without failure. Similarly Server 4 and 5 are in different regions and in same the same VLAN.
- **Servers of same VLAN can communicate to each other directly**
- Servers 1, 2 and 3 can communicate to each other directly via virtual switch. For internal communication they don't have to cross the VMware. Similar case for Server 4 and 5.

Designing virtual network security

Designing a secure virtual network involves establishing a security baseline, segmenting the network, implementing firewalls, and monitoring activity. Key principles include the principle of least privilege, limiting public IP addresses, and regularly updating security settings.

- 1. Establish a Security Baseline: Review existing security baselines and incorporate applicable measures. Define security objectives and requirements for the virtual network.
- 2. Segmentation: Use subnets to isolate different types of workloads and traffic. Implement Network Security Groups (NSGs) as firewalls at the subnet level to control traffic flow. Route external traffic through a Network Virtual Appliance (NVA), such as a firewall, for monitoring and control. Use Fully Qualified Domain Names (FQDNs) to filter internet access.
- 3. Firewalls: Implement firewalls to block unauthorized access and control traffic flow. Consider deploying Network Virtual Appliances (NVAs) like firewalls for enhanced security.
- 4. Monitoring and Auditing: Monitor network activity to detect suspicious behavior and take appropriate action. Conduct regular audits to assess security posture and identify vulnerabilities.
- 5. Principle of Least Privilege: Configure role-based access control (RBAC) to grant users only the necessary permissions. Ensure users only have access to the resources and settings required for their job function.

Key Design Principles:

- **Security Objectives:** Define clear security goals, including data confidentiality, integrity, and availability.
- **Segmentation:** Divide the network into logical segments (subnets) based on security requirements, using NSGs to control traffic flow between them.
- **Least Privilege:** Grant users and resources only the necessary permissions, minimizing the potential impact of security breaches.
- **Defense in Depth:** Implement multiple layers of security controls, including firewalls, intrusion detection systems, and encryption.
- **Monitoring and Auditing:** Continuously monitor network activity for suspicious behavior and audit security configurations.
- **Security Measures:**
 - **Network Security Groups (NSGs):** Utilize NSGs as firewalls at the subnet level to filter traffic based on rules that allow or deny access.
 - **Network Virtual Appliances:** Deploy firewalls and other security appliances within the virtual network to filter traffic and provide advanced security features.
 - **Public IP Addresses:** Minimize the use of public IP addresses and consider using shared IP addresses from services like Azure Front Door.
 - **Encryption:** Encrypt sensitive data in transit and at rest to protect it from unauthorized access.
 - **Access Control:** Implement role-based access control (RBAC) to manage user permissions and limit who can modify network configurations.
 - **User Accounts and Permissions:** Create user accounts and assign appropriate permissions to control access to network resources.

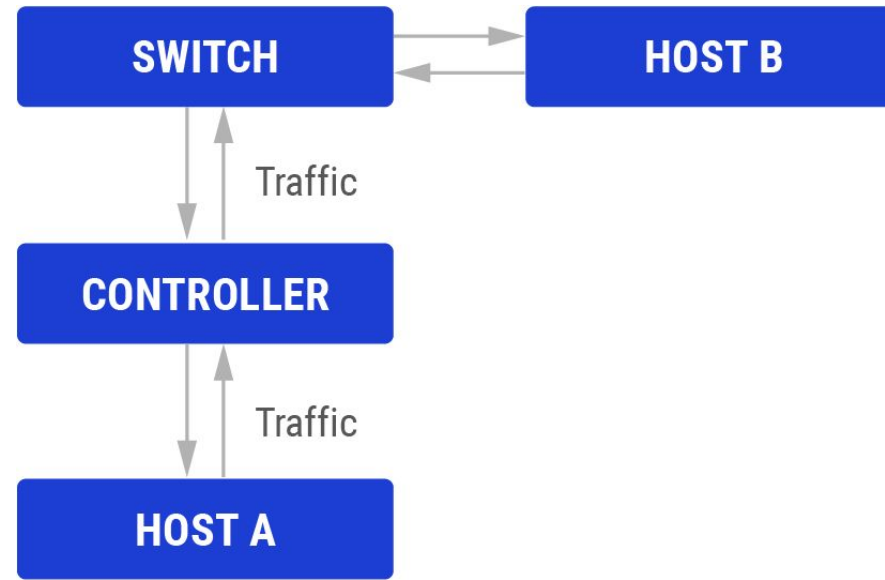
Software-defined security

Software-defined security (SDS) is a security model where information security is managed through software, rather than relying solely on hardware-based security devices. This approach allows for centralized control, policy enforcement, and automation of security functions like firewalls, intrusion detection, and access control. SDS leverages software-defined networking (SDN) to enhance network security and provides flexibility, scalability, and visibility in security management.

The key features and attributes of SDS

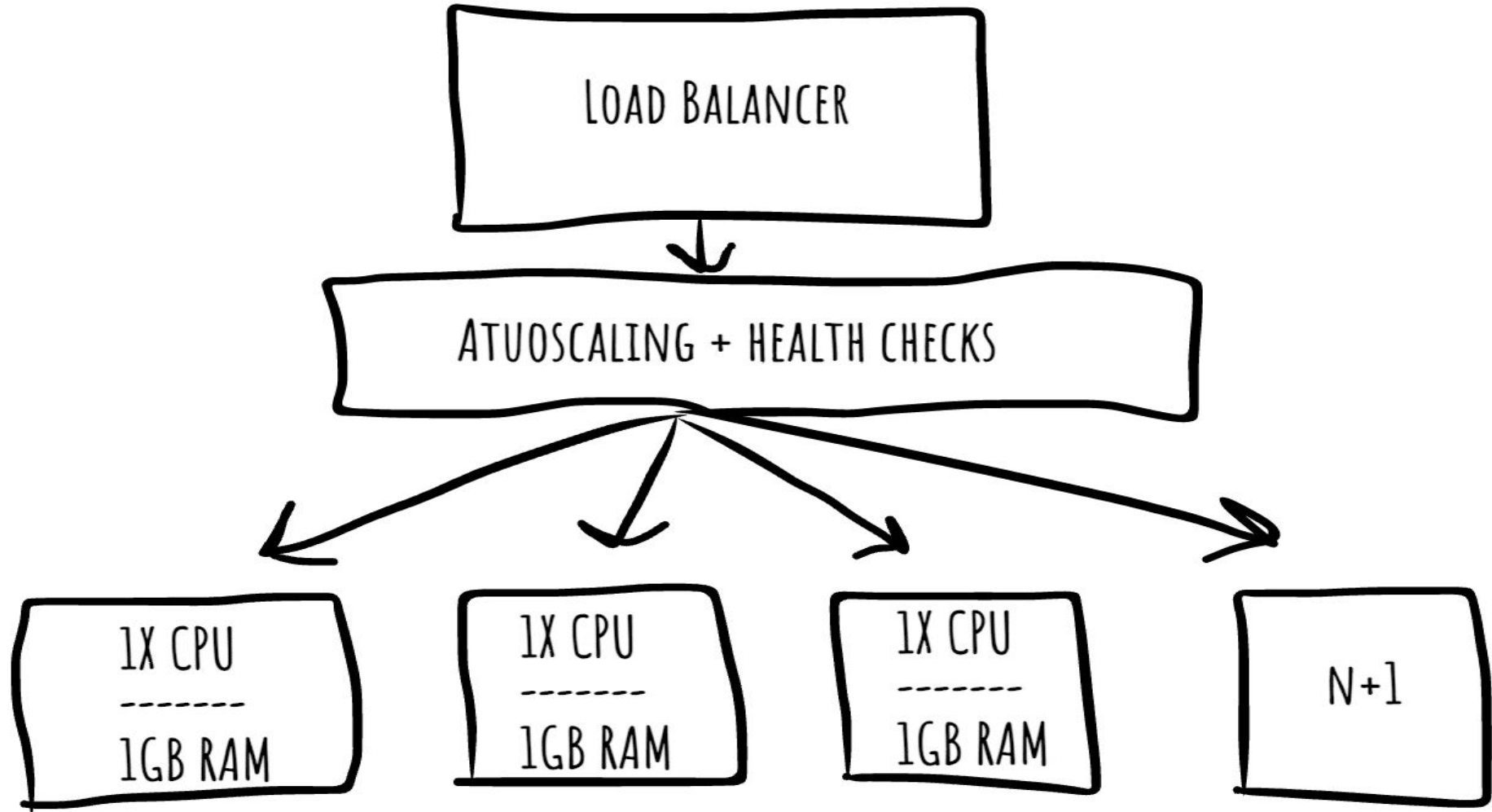
- **Abstraction:** SDS abstracts security policies from the hardware layer. They are run within the software layer. Common security models are defined and can be deployed repeatedly.
- **Automation:** each asset or device in the system is deployed and put in a security trust zone automatically. This is performed manually in traditional security approaches.
- **Flexibility:** SDS is entirely software-based. The security policy is flexible, and security is available “on demand”. It is relatively easy to scale up and adapt to changes. It consistently enforces security policies across board irrespective of the location of the network systems.
- **Concurrency control:** network security controls (such as intrusion and prevention, network segmentation, firewalling, violation monitoring, etc.) work together concurrently. Such an orchestration improves security and reduces the cost.
- **Visibility:** because security can be virtualized, network professionals can discover abnormal activities that would not be possible with physical devices.
- **Portability:** SDS allows for assets to carry their security settings with them as they scale or move from one location to another.

- Three components are integrated to protect the network, as schematised in the diagram above. They are the enablers of SDS.



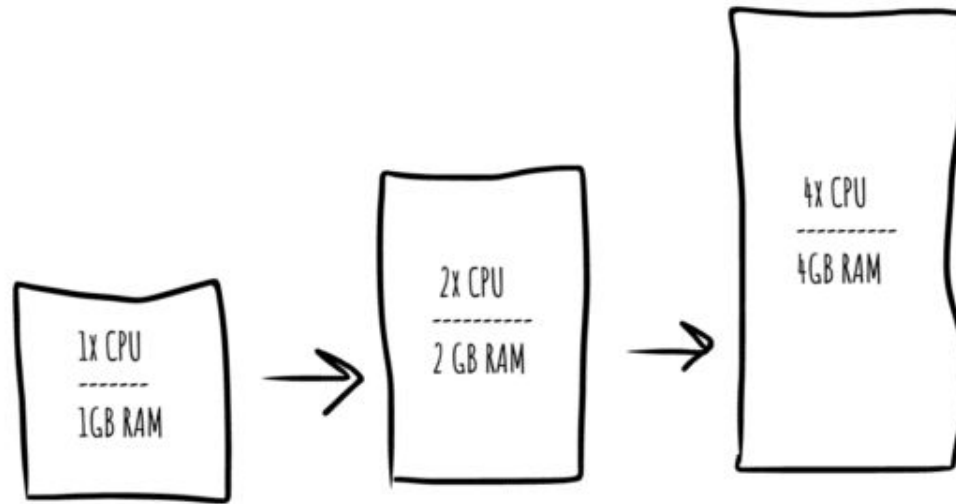
- **Host:** the host aims to send or receive data through the network. In traditional networks, security techniques reside in the host. The host checks new packets to see if they have threats. For the SDS, all security techniques are transferred to the controller.
- **Controller:** the controller is fully software based. All security checks are done inside the controller. The controller must have efficient access control by stating which types of packets should be carried within the network. It has visibility of the traffic flows and collects and processes information about the network.
- **Switch:** the switch consults the controller to decide whether to accept or reject a request. The current switches have limited storage capacity and cannot store all the rules. A reactive caching mechanism is adopted in SDN. However, this makes switches vulnerable to Denial of Service (DoS) attacks

Antifragile Cloud Architecture



Fragile: Vertical Scaling

- Something fragile can only withstand so much stress before hitting a breaking point with no return to stability without outside intervention.
- Vertical scaling is the response to stress of adding more computing resources to a single server. This could mean upgrading to a processor with more cores or adding more memory to the server. Essentially, better hardware means more capacity for handling workloads.



This technique is fragile because it is non-redundant. That means when a vertically-scaled server is using all its resources, users will experience degraded performance, if not a complete outage. And adding more computing resources is usually not something you can do on-the-fly as it requires provisioning a new server with more resources and then cutting over the stressed server to the new one.

Resilient: Horizontal Scaling

Something resilient is capable of bouncing back from stress and returning to a stable baseline. Horizontal scaling is the treatment where server hardware resources typically stay the same (as opposed to vertical scaling). However, more servers are added behind a load balancer to process the workload. In this scenario, capacity increases with the number of servers.

You can consider this type of scaling resilient because it is redundant. Redundancy is usually a good thing to have in the event of disorder.

So, in the usual event of something unusual happening, a horizontal system can more easily recover than that of vertical scale. Take, for example, a web request that triggers an unintentional memory leak on the server, leading to out-of-memory errors. In the single vertical scale architecture, the server will seize up. It will no longer be able to process work due to no memory, causing an outage. The horizontal architecture will suffer less harm because it will only seize up one server of the many, allowing work to continue.

Scaling horizontally will still have a capacity limit just as vertical scale does. However, it is usually easier to add capacity to this architecture because the server unit is predefined, you just have to add more of them to increase capacity. There is no figuring out processor cores, amount of memory, etc. It's more or a matter of simple math. For example, if there are two servers and the current workload is using 100% of capacity, add one more server unit, and you'll add 50% more capacity.

Antifragile: Horizontal Autoscaling

Antifragile cloud architecture will still experience harm from stress, but it will bounce back and benefit from it in the long run.

Autoscaling is an automated extension of horizontal scaling. Instead of manually adding servers to increase capacity, it can be done automatically.

The system will be self-healing and not require manual intervention based on throughput levels. This means your applications could experience a 10 times (or more) burst of traffic, and you can be sipping mezcal oceanside without the worry of the application going down due to traffic (in theory). Or, even if a server experiences an unusual error, auto-scaling architecture would know how to handle a bad server unit and replace it with a healthy one. That's antifragile cloud architecture.

Autoscaling does have its limits, however. There is usually a point where a supporting system maxes out. This puts a cap on autoscaling, requiring intervention. You can experience benefits from scaling out up until the point of your next bottleneck. This could be a database, key-value store, load balancer, etc. For example, if a database is running at 100%, adding more capacity to handle web requests will not add benefit. that's because web requests can only be processed as fast as the slowest systems in the process, or the database in this example.

Examples of Antifragile Techniques:

- **Load Balancing:**
 - Distributing traffic across multiple servers to prevent a single point of failure from causing a system outage.
- **Automated Backups and Recovery:**
 - Regularly backing up data and having automated recovery processes in place can ensure that the system can be quickly restored after a failure.
- **Redundancy:**
 - Creating redundant systems or components to ensure that if one fails, another can take over.
- **Microservices Architecture:**
 - Breaking down applications into smaller, independent services that can be deployed and updated independently, making it easier to isolate and resolve failures.
 - Antifragile cloud infrastructure can be designed to be more robust, adaptable, and resilient to unexpected events, ultimately leading to improved performance.

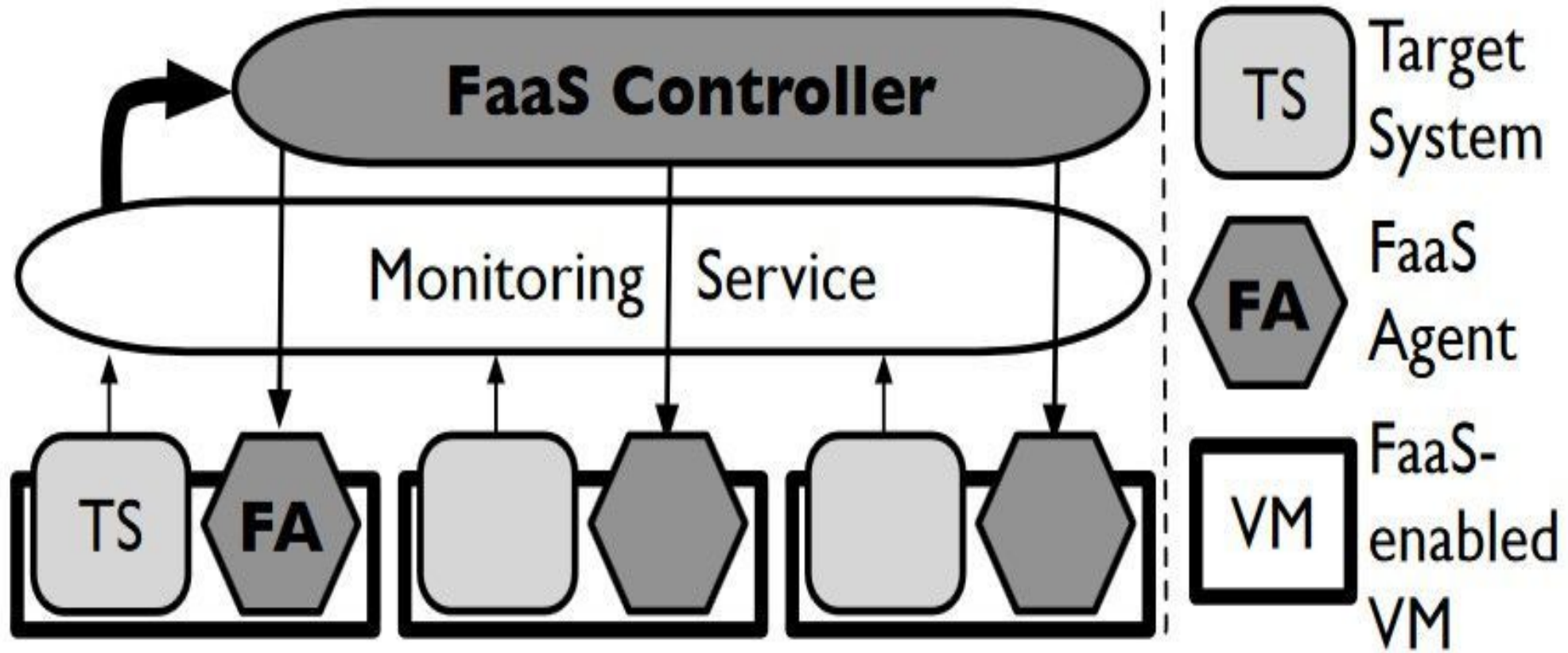
Failure as a Service(FaaS)

Failure as a Service (FaaS) is a cloud service that enables the controlled and deliberate introduction of failures into a system for testing and resilience purposes. It allows cloud services to perform large-scale failure drills in real deployments, helping to improve the robustness and reliability of cloud applications. FaaS is also known as Resiliency as a Service or controlled disruption.

A group of researchers from the University of California at Berkeley has offered a basic FaaS model for introducing common failures into large-scale distributed cloud services. This architecture includes three main components:

- FaaS controller — A fault-tolerant service that sends failure commands to the agents that run in VMs.
- FaaS agent — An agent that runs on the same VM as the tested service, receiving failure commands from the controller and initiating controlled disruptions
- Monitoring service — A tool that monitors the behavior of target services and gathers information needed for creating failure drill scenarios and specifications

FaaS Architecture



- FaaS engineers control disruptions in the system via a set of carefully designed experiments. Each experiment recreates a potential failure that allow to analyze the system's behavior and determine the most effective ways of handling this particular issue. In this way, we can constantly improve the disaster recovery program and significantly shorten the time needed for system recovery if one of these failures were to occur in real life.

Failure testing consist of the following steps:

- Define the normal state or behavior of the target system.
- Introduce a disruption similar to real-world events such as a server crash or network connection failure.
- Watch the performance of the target system and see if it can stay stable while the failure drill is running.

Common Failure Scenarios Tested with FaaS:

- **Network Outages:**

- Simulating network interruptions to test the system's ability to handle connectivity failures.

- **Resource Depletion:**

- Simulating resource limitations (e.g., memory, CPU) to test the system's ability to handle resource constraints.

- **Application Crashes:**

- Simulating application failures to test the system's ability to recover from unexpected shutdowns.

- **Data Corruption:**

- Simulating data corruption to test the system's ability to recover from data loss.